



Working with Text Data

Chapter 2: From Raw Text to Token Embeddings

Juan Irving Vasquez

October 7, 2026

Instituto Politécnico Nacional (IPN)

The LLM Preprocessing

- **Input:** Raw text (books, websites, code).
- **Goal:** Convert discrete linguistic symbols into numerical tensors suitable for deep learning.
- **Vectorization:**
 1. Text normalization and cleaning.
 2. Tokenization: Breaking text into smaller units.
 3. Mapping: Converting tokens to integer IDs.
- **Embedding:**
 - Projecting IDs into a continuous vector space.

Vectorization

- Deep learning models, being differentiable functions, can only process numeric tensors
- They can't take raw text as input.
- Vectorizing text is the process of transforming text into numeric tensors.

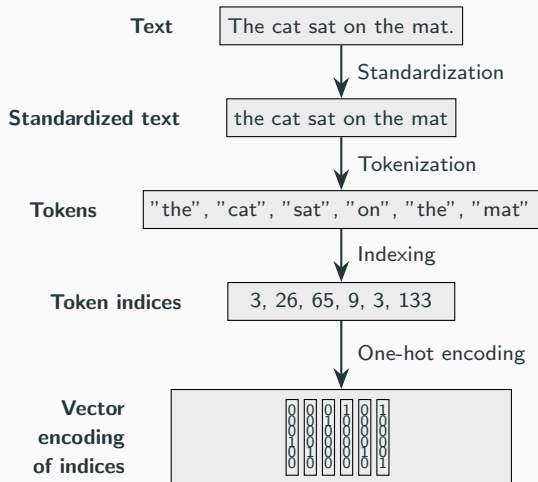


Figure 1: Data vectorization.

There are two kinds of text-processing models:

- **Sequence models:** care about word order (uses word-level tokenization).
- **Bag-of-words models:** treat input words as a set, discarding their original order (uses N-gram tokenization).

N-grams Example

"the cat sat on the mat." $\rightarrow$ {"the", "the cat", "cat", "cat sat", "sat", "sat on", ...}

Tokenization

There are different levels for decomposing text:

- **Word-level tokenization:** tokens are space-separated (or punctuation-separated) substrings.
- **N-gram tokenization:** tokens are groups of N consecutive words (e.g., "the cat" is a 2-gram or bigram).
- **Character-level tokenization:** each character is its own token.
- **Subword-level (Standard for LLMs):**
 - Examples: **Byte Pair Encoding (BPE)**, WordPiece.
 - Breaks rare words into sub-parts (e.g., "unhappy" → "un", "happy").
 - Balance between vocabulary size and sequence length.

Vocabulary Indexing

- Once text is split into tokens, you need to encode each into a numerical representation.
- In practice, you build an index of all terms in the training data (the "vocabulary") and assign a unique integer to each entry.

```
vocabulary = {}  
for text in dataset:  
    text = standardize(text)  
    tokens = tokenize(text)  
    for token in tokens:  
        if token not in vocabulary:  
            vocabulary[token] = len(vocabulary)
```

Byte Pair Encoding (BPE) Mechanism

BPE is the algorithm used by GPT-2, GPT-3, and Llama.

1. Start with individual characters as the initial vocabulary.
2. Iteratively merge the most frequently occurring adjacent pair of tokens into a new single token.
3. Repeat until a target vocabulary size is reached.

Advantages:

- Efficiently handles unknown words by falling back to character-level components.
- Reduces the total number of tokens per sequence compared to character-level.

The Role of Special Tokens

To help the model understand structure, we introduce non-text tokens:

- `<|endoftext|>`: Marks the end of a document or sequence (crucial for GPT-style models).
- `<|unk|>`: (Less common in BPE) Represents words not in the vocabulary.
- `<|pad|>`: Used to ensure sequences in a batch have the same length.

Context Window: The maximum number of tokens an LLM can process at once (e.g., 2048, 4096, or 128k).

Sliding Window for Training

LLMs are trained to predict the *next* token. We use a sliding window to create input-target pairs.

```
# Input:  [1, 2, 3, 4, 5]
# Window size = 4
x = [1, 2, 3, 4] # Input sequence
y = [2, 3, 4, 5] # Target sequence (shifted by 1)
```

- For each position i in the input, the model learns that the ground truth is the token at $i + 1$.

Token Embeddings

- **Identity Mapping:** Integer IDs carry no semantic meaning (ID 5 is not "closer" to ID 6).
- **Embedding Layer:** A lookup table of size ($VocabSize \times EmbeddingDimension$).
- **Projection:** Each token ID is mapped to a vector in $\mathbb{R}^d$ (e.g., $d = 768$ for GPT-2).
- These vectors are **learnable parameters** updated during backpropagation.

Adding Order: Positional Embeddings

Since the Transformer architecture is permutation-invariant (it processes all tokens in parallel), it doesn't "know" the position of a token.

The Solution

Add (or concatenate) a position-dependent vector to the token embedding.

- **Absolute Positional Encodings:** Fixed sine/cosine functions or learnable vectors for each index $[0, 1, \dots, MaxContext]$.
- **Relative Positional Encodings:** Focus on the distance between tokens (used in more recent architectures like RoPE).

Input Tensors for PyTorch

For a batch of text, the input tensor usually has the shape:

[BatchSize, ContextLength, EmbeddingDimension]

- **Batching:** Processing multiple sequences in parallel to maximize GPU utilization.
- **Padding:** If sequences have different lengths, they are padded to match the longest one in the batch.

Next Chapter: Coding Attention Mechanisms